BoothNet

DreamBooth-LoRA trained with a frozen depth ControlNet in the loop

Technical report: data, models, preprocessing, training, inference, status and ideas

ELEC 542 final project · Pandula Thennakoon and Vishal Venkat · status as of November 13, 2026

Phase	Content	Status
0 Setup	conda env, check_env.py	done
1 Data	preprocessing (6 subjects), test bank (75 cases)	done
2 Training	train.py, M0–M3 configs, dog M0 + M1 trained	partly done (2 of 24 runs; 22 planned on a server)
3 Inference	generate.py, scale/checkpoint sweep, full generation	not started (exploratory sets only)
4 Evaluation	evaluate.py, aggregate.py, tables, plots	not started

How to read this document. Sections 1–2 give the idea and the end-to-end pipeline. Sections 3–4 list the inputs and every pretrained model. Sections 5–6 explain how `dataset/` becomes `data/` (preprocessing and test bank). Section 7 is the training method in detail, section 8 the variants (M0–M3) and baselines, and section 9 how inference works. Section 10 shows the results so far. Sections 11–13 list decisions, known issues and improvement ideas. Section 14 maps every file and function. File paths refer to the repository root.

Contents

1 Idea and hypotheses
2 End-to-end pipeline
3 Data inputs
4 Pretrained models
5 Data preparation (dataset/ → data/processed)
6 Test control bank (data/test_bank)
7 Training (train.py)
8 Variants M0–M3 and evaluation methods
9 Inference
10 Results so far (dog)
11 Decisions and deviations from the brief
12 Known issues and limitations
13 Ideas for improvement
14 Code map, commands and configuration reference

1 Idea and hypotheses

Problem. DreamBooth fine-tunes a text-to-image model on 3–5 photos of one subject so that a rare token (here `sks`) plus a class noun ("a photo of sks dog") generates that subject. With so few photos, the fine-tuned weights must explain *everything* in them with the token: appearance, but also pose, viewpoint and background. The model therefore memorizes training poses and backgrounds. ControlNet adds spatial control (here a depth map) to a frozen Stable Diffusion model. Combining the two naively (train DreamBooth, attach ControlNet only at inference) works reasonably well, because ControlNets transfer to fine-tuned models.

BoothNet's idea. Put the frozen, pretrained depth ControlNet *inside the LoRA training loop*. Each training photo is denoised with its own (degraded) depth map as a hint. Because the depth map already explains the geometry (pose, shape, layout), the LoRA should only need to learn what depth cannot provide: color, texture and markings. At test time a depth map from any new scene supplies new geometry, and the LoRA paints the subject into it. Four tricks keep this from going wrong: control dropout, depth degradation, random conditioning strength, and an optional background-down-weighted loss (section 7.4).

Research question. Does training the DreamBooth-LoRA with the ControlNet in the loop give a LoRA that (a) follows new layouts better and (b) keeps identity at least as well, compared with training without it?

Hypothesis	Statement	How it will be tested
H1	Proposed (M1/M2) has better layout fidelity than the naive combo (M0 + ControlNet) at equal identity, especially when the test layout is far from the training poses.	Depth RMSE and mask IoU vs DINO-masked identity, Tier B/C, scale sweep curves
H2	Control dropout keeps text-only generation (no ControlNet) close to M0; always-on control (M3) hurts it.	Text-only sanity table: B1 vs M1/M2/M3 (DINO, CLIP-T)
H3	The benefit is larger for live subjects (pose = layout) than for rigid objects (silhouette = identity).	All tables split by category (live / object)

2 End-to-end pipeline

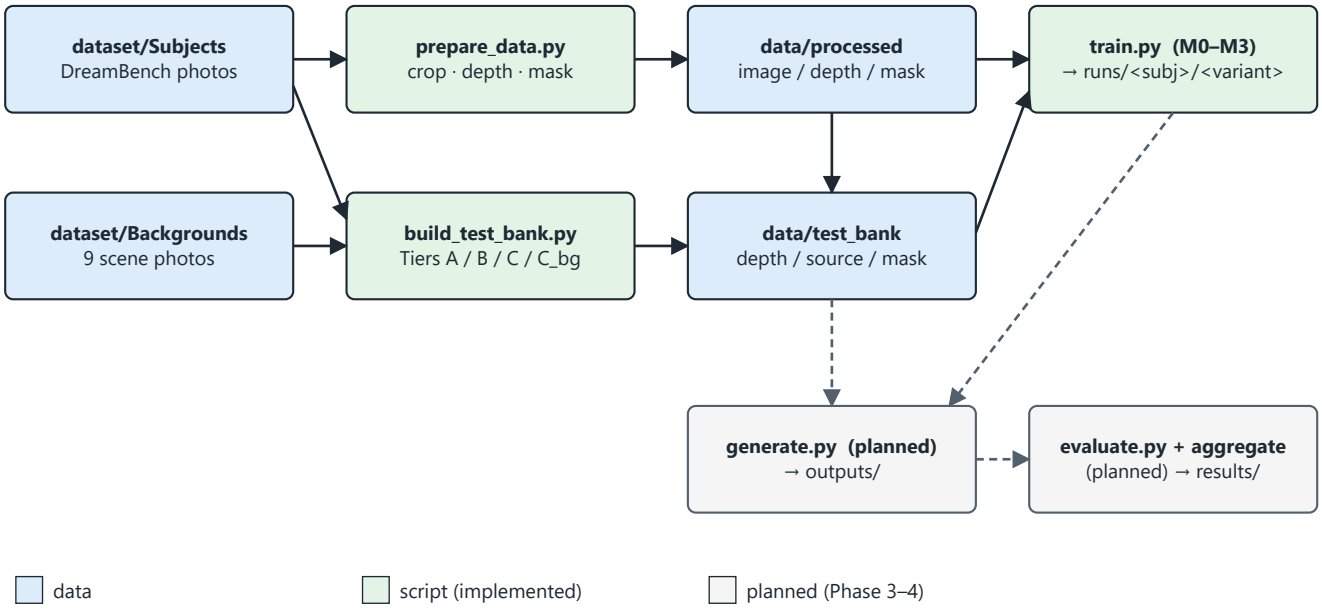


Figure 1. Data flow. Green boxes are implemented scripts, grey boxes are planned (Phases 3–4).

Folder	Written by	Contents	In git
dataset/	user (read-only)	DreamBench subject photos, prompts_and_classes.txt, background scenes	no
data/processed/	prepare_data.py	per subject and photo: 512^2 image, depth, mask, meta.json; split.json; preview.png	no
data/test_bank/	build_test_bank.py	test cases per subject and tier: depth, source image, source mask, case.json	no
runs/	train.py	LoRA checkpoints, resolved config, training log, validation images, loss plot	no
outputs/	check_env, exploration (later generate.py)	generated images	no
results/	evaluate.py / aggregate.py (planned)	metrics CSV, tables, plots	no
configs/	hand-written	subjects, backgrounds, test bank, training base + variants	yes

3 Data inputs

3.1 Subjects (DreamBench)

`dataset/Subjects/` is the DreamBooth dataset (DreamBench): 30 subjects, 4–6 photos each, plus `prompts_and_classes.txt`, which gives the class noun of each subject (used to check the configured classes). Six subjects are used, configured in `configs/subjects.yaml`:

Subject	Class noun	Category	Photos	Train	Held-out	Notes
dog	dog	live	5	00–03	04	corgi; close-up portraits

Subject	Class noun	Category	Photos	Train	Held-out	Notes
dog6	dog	live	5	00–03	04	corgi puppy, studio backgrounds
cat2	cat	live	5	00–03	04	grey cat; 00/01/04 have baked-in black bars
backpack	backpack	object	6	00–04	05	00/01 worn by a person
teapot	teapot	object	5	00–03	04	often with saucer / cups / stand
rc_car	toy	object	5	00–03	04	segmentation prompt "toy car"

The held-out photo is the last file in sorted order. It is never used for training. It serves as the Tier A test case and as a validation input. Other DreamBench folders are used only as Tier B sources (other instances of the same class: dog2, dog3, dog7, cat, backpack_dog).

3.2 Backgrounds

`dataset/Backgrounds/` holds 9 user-provided scenes (1280^2 except beach2, 2000×1500). Six are empty scenes, used to build composite depth maps. Three contain a cat, used directly as "scene" test cases for the cat subject: can the personalized cat be generated in another cat's pose?

File	Type	Prompt context	Used for
beach1.jpg	composite	on the beach	all 6 subjects
beach2.jpg	composite	on a pebble beach by the sea	all 6 subjects
city_roads.jpg	composite	on the street of a mountain town	all 6 subjects
city_sunset_new_york.jpg	composite	on a city street at sunset	all 6 subjects
road-mountains.jpg	composite	on a road with snowy mountains	all 6 subjects
times_square1jpg.jpg	composite	in Times Square	all 6 subjects
cat_pose1.jpg	scene (cat)	sitting on the grass	cat2 only
cat_pose2.jpg	scene (cat)	leaning over a wall (watercolor painting)	cat2 only
cat_pose3.jpg	scene (cat)	lying on a rug	cat2 only

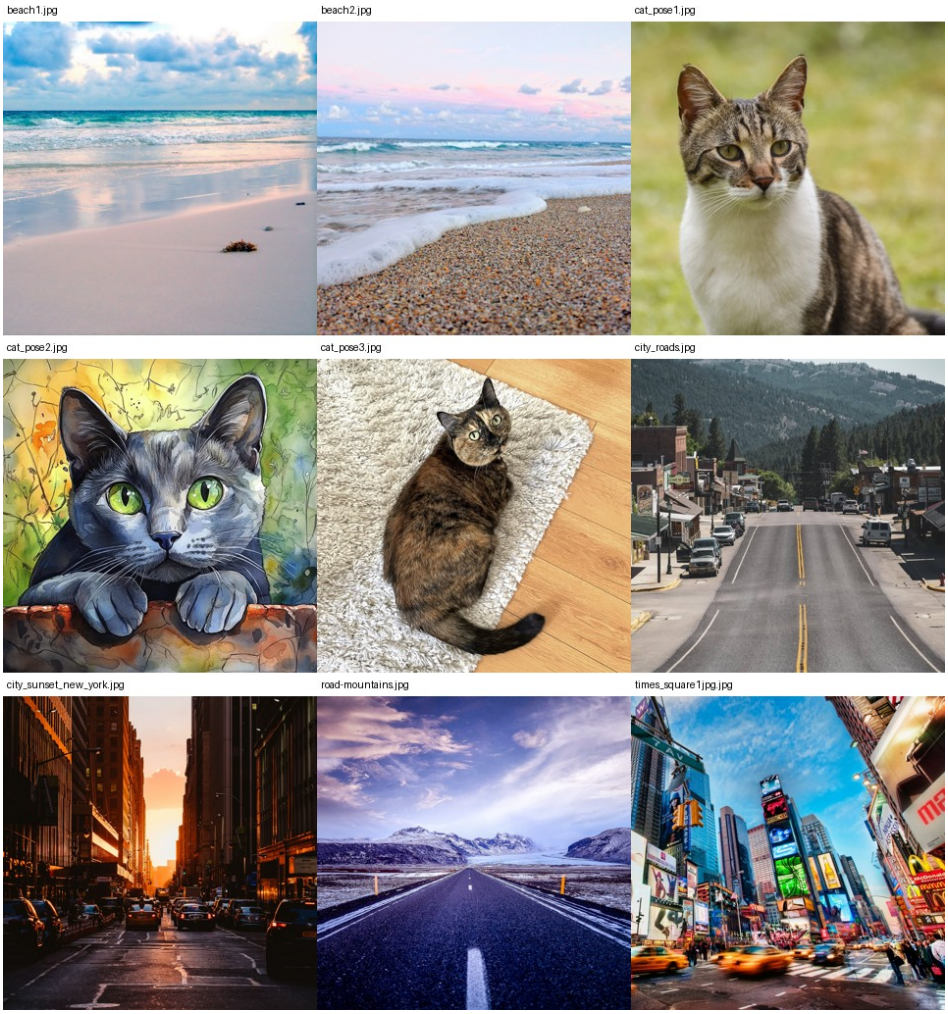


Figure 2. The 9 background images (center-cropped).

4 Pretrained models

No model is trained from scratch. The only trainable parameters in the whole project are the LoRA adapters (3.19 M parameters) inside the SD1.5 UNet. Everything else is frozen.

Role	Model (Hugging Face ID)	Used in	Precision / notes
Base text-to-image model	stable-diffusion-v1-5/stable-diffusion-v1-5 (VAE, CLIP ViT-L/14 text encoder, UNet ~860 M, DDPM noise schedule)	training, validation, inference, Tier B generation	bf16 frozen in training; fp16 at inference
Layout control	lllyasviel/control_v11f1p_sd15_depth (ControlNet v1.1 depth)	training (M1–M3), validation, inference	frozen; trained on MiDaS-style depth, near = bright
Depth estimator	depth-anything/Depth-Anything-V2-Small-hf	all depth maps (training, test bank; planned depth-error metric)	fp16; relative inverse depth
Open-vocabulary detector	IDEA-Research/grounding-dino-base	masks (box for the class noun), subject-aware crop, Tier B seed filter	fp32; box threshold 0.30, text 0.25

Role	Model (Hugging Face ID)	Used in	Precision / notes
Segmenter (objects)	facebook/sam-vit-large (Segment Anything)	object masks from the detector box	fp32
Salient segmenter (animals)	BiRefNet, rembg session "birefnet-general" (ONNX)	animal masks	onnxruntime-gpu; ~9 GiB VRAM
Identity metric (planned)	facebook/dino-vits16	DINO / DINO-masked similarity	Phase 4
Image–text metric (planned)	openai/clip-vit-base-patch32	CLIP-I, CLIP-I-masked, CLIP-T	Phase 4
Optional baseline	Salesforce/blipdiffusion-controlnet	B4 (only if time permits)	not used yet

Schedulers: training uses SD1.5's DDPM noise schedule (1000 steps, scaled-linear β from 0.00085 to 0.012, ε-prediction). Validation and inference use the UniPC multistep sampler (30 steps) built from the same config.

5 Data preparation (dataset/ → data/processed)

Script: `scripts/prepare_data.py` (logic in `boothnet/preprocess.py`). For every photo of every configured subject it produces a 512×512 image, a depth map and a binary subject mask. It runs in two stages because of GPU memory (5.4).

5.1 Steps per photo

- **Strip black bars** (`strip_borders`): rows and columns whose maximum gray value is below 16 are removed if the bar is at least 8 px. Some DreamBench files have letterbox bars baked in (cat2 00/01/04: 320 px on 1920^2 files; teapot 03: 12 and 45 px). Without this the LoRA would learn black bars.
- **Square crop** (`crop_for_subject`, `square_crop_resize`): square images are untouched. An image that is non-square after bar stripping gets a *subject-aware* crop: Grounding-DINO finds the subject box (on a copy with long side 1024), and the square window keeping the most box pixels is chosen, ties broken toward the center. Without a detection the crop is centered. This affects 4 images and keeps 89–100% of the cat in cat2.
- **Resize** to 512×512 (bicubic) → `image.png`. No horizontal flips anywhere: subjects have asymmetric markings.
- **Depth** (`DepthEstimator`): Depth-Anything-V2-Small predicts relative inverse depth (larger = nearer). It is upsampled (bicubic) to 512^2, min-max normalized per image to [0,1] (near = bright, the convention the SD1.5 depth ControlNet expects), and saved as 8-bit 3-channel `depth.png`.
- **Mask** (`SubjectMasker`, 5.3) → binary `mask.png` (0/255).
- **Metadata**: `meta.json` records subject, class, category, split, source file, crop boxes, crop mode, and mask method / detection box / score / prompt.

5.2 Outputs

```
data/processed/<subject>/
    split.json              {"subject", "class", "category", "train": [...], "heldout": ["04"]}
    preview.png             contact sheet: image | depth | mask | overlay (check by eye)
    <img_id>/image.png      512x512 RGB
    <img_id>/depth.png      512x512, 3 equal channels, near = bright
    <img_id>/mask.png       512x512, 0 / 255
    <img_id>/meta.json
```

5.3 Subject masks: why a hybrid, and how it works

Masks are used for M2's masked loss (training), for the Tier C composites, and, with the same masker, for the planned evaluation metrics (DINO-masked, CLIP-I-masked, mask IoU). Two off-the-shelf approaches each failed on part of the data:

- **Salient-object segmentation (BiRefNet)** segments whatever is salient. It got animals right but included the person wearing the backpack, and the teapot's saucer, cups and stand.
- **Class-prompted segmentation (Grounding-DINO box → SAM)** isolates the named object, but on close-up corgi portraits Grounding-DINO's "dog" box covers only the head, so SAM drops the white body.

So `SubjectMasker` picks the method by category:

Category	Procedure
object	Grounding-DINO box for the segmentation prompt (class noun; "toy car" for rc_car) → SAM-ViT-L with the box as prompt, 3 candidate masks. Chosen: the **largest** candidate with <2% of its area outside the box and predicted IoU > 0.7 (fallback: highest predicted IoU). Pure argmax-IoU dropped the driver figure of rc_car/04; the box constraint keeps context (saucers, people) out. Then fill enclosed holes <0.1% of the image.
live	BiRefNet salient mask (threshold 0.5) → connected components (8-connectivity); keep a component if its area is ≥0.2% of the image and ≥10% of it lies inside the Grounding-DINO box for the class noun. This keeps the whole animal and rejects unrelated salient things in busy generated scenes (cars, people). Then fill small holes.
both	No detection → empty mask. In evaluation this will count as a segmentation failure (informative in itself).

5.4 GPU memory: two-stage processing

BiRefNet's ONNX graph needs about 9 GiB of GPU memory and cannot share a 12 GB GPU with the torch models. `SubjectMasker` therefore works in two stages: (1) all torch work (crop, depth, detection, SAM for objects), then release Grounding-DINO and SAM; (2) open one BiRefNet session, compute all animal masks, close the session. `mask_batch(images, texts, categories)` wraps both stages for later use in evaluation. Note: the ONNX runtime must be the CUDA-12 build (`onnxruntime-gpu==1.23.2`). Version 1.31 targets CUDA 13 and silently falls back to the CPU (~2 min per image).



Figure 3. Preprocessed dog (left) and backpack (right): image, depth (near = bright), mask, overlay (background tinted red). Held-out photo in the last row. Backpack masks exclude the person.

6 Test control bank (data/test_bank)

Script: `scripts/build_test_bank.py` with `configs/test_bank.yaml` and `configs/backgrounds.yaml`. A test case is a layout to generate into. It is a depth map plus a source image, a source mask (for IoU) and a prompt context, organized in tiers of increasing distance from the training photos. 75 cases in total.

Tier	Depth map from	Prompt	Per subject	Purpose
A	the subject's held-out photo	"a photo of sks <class>"	1	unseen photo, same subject: easiest
B	another instance of the class (real photo) or base-SD1.5 image	"a photo of sks <class>"	3	new pose / shape from a different individual
C (composite)	background depth + pasted subject-shaped depth	+ background context	6	new scene + placement
C (scene)	cat background photo (contains a cat)	+ context	3 (cat2)	subject in another cat's pose

Tier	Depth map from	Prompt	Per subject	Purpose
C_bg	background depth only (nothing pasted)	+ context	6 (dog, backpack)	exploratory: does the model insert the subject itself?

6.1 Tier B sources

- dog and dog6: dog2/00 (chow chow walking toward camera), dog3/03 (standing, side view), dog7/01 (lying).
- cat2: cat/00, cat/01, cat/04. backpack: backpack_dog/01, 02, 04.
- teapot and rc_car have no other instance. Base SD1.5 (no LoRA, no ControlNet; UniPC 30 steps, CFG 7.5) generates "a photo of a teapot" / "a photo of a toy car". Seeds 0–15 are tried in order, keeping the first 3 whose Grounding-DINO box doesn't touch the border (a cut-off object can't serve as a layout). Kept: teapot seeds 0, 2, 3; toy car seeds 5, 6, 8. *Deviation:* rc_car's class noun is "toy", but "a photo of a toy" yields arbitrary toys, so Tier B uses "toy car".

6.2 Tier C composite depth (composite_depth)

Pose source: the subject's Tier A photo, unless its mask touches the image border (then the subject can't stand on the ground), in which case the first un-truncated Tier B case. Result: dog and dog6 use dog2_00, cat2 uses cat_01, the three objects use their own held-out photo. Placement defaults (per-background overrides possible):

- crop the pose source's depth and mask to the mask's bounding box;
- scale to height `size_frac` × 512 = 230 px (aspect kept, width capped at 90% of the image), depth bilinear, mask nearest;
- place bottom-center: horizontal center `x_center` = 0.5, lowest pixel at `y_bottom` = 0.92 × 512;
- b = median background depth in a 6-row strip at the feet; pasted depth is rescaled linearly from its own [min, max] into [min(b + δ, 0.95), 1] with δ = 0.05, so the subject is always nearer than the ground it stands on;
- the pasted mask becomes the case's source_mask (for mask IoU).

Example (dog, beach1): feet depth b = 0.871, pasted range [0.921, 1.0], box (186, 241)–(325, 471). Because the ground at the feet is already near, the subject's internal relief is compressed into ~[0.9, 1]: an almost flat white silhouette. This is physically consistent, but it carries less 3-D detail (see section 12).

```
data/test_bank/<subject>/<tier>/<case_id>/
    depth.png  source.png  source_mask.png  case.json   (+ overlay.png for Tier C)
case.json: subject, class, category, tier, case_id, context,
           prompt_personalized ("a photo of sks dog on the beach"), prompt_class ("a photo of a dog on the beach"),
           and per tier: source_ref / crop / mask info / placement / bg_depth_at_feet / pose_source ...
```



Figure 4. Tier A and Tier B cases (source tinted by its mask | depth), two cases per row.

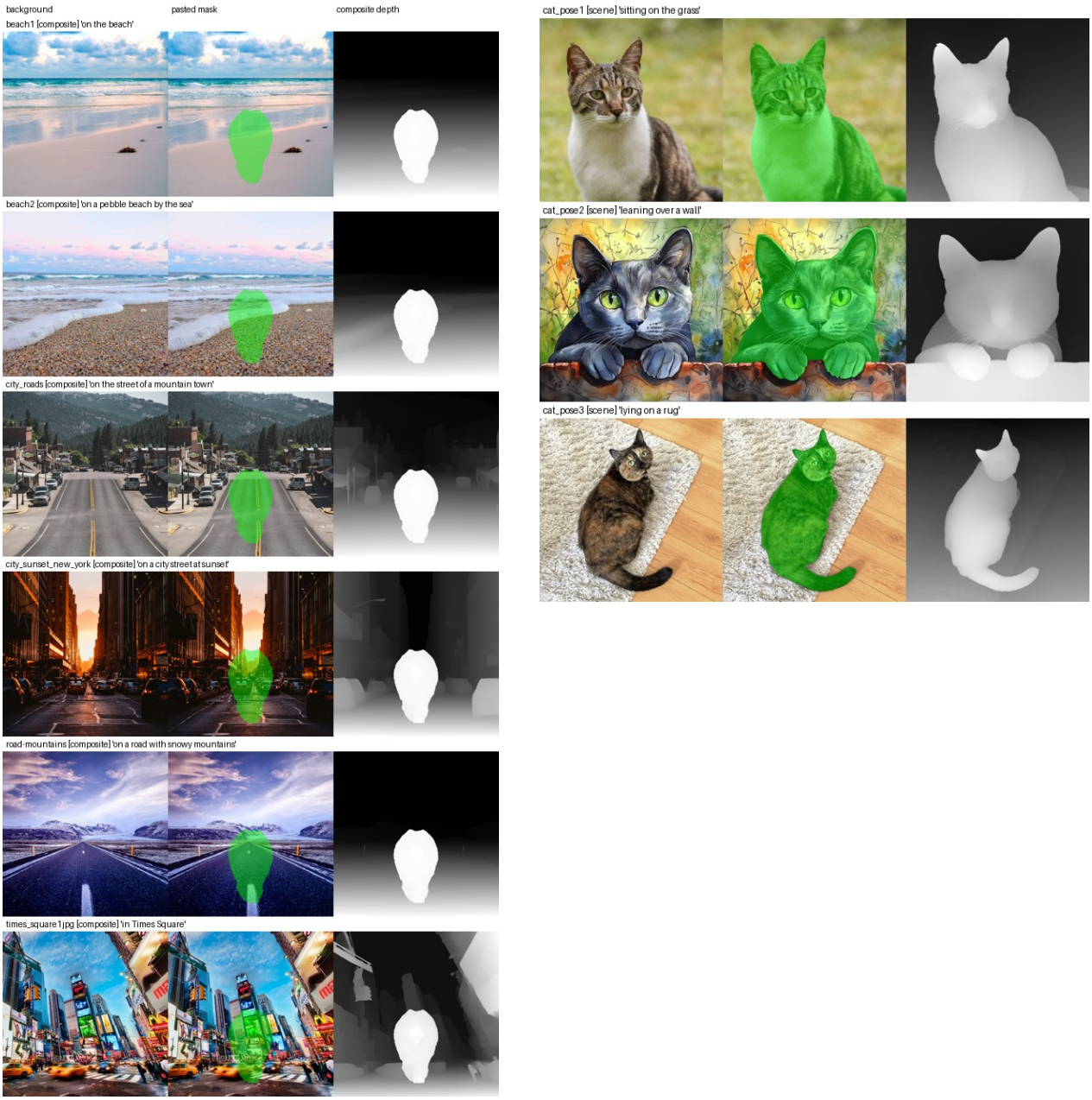


Figure 5. Left: dog's six Tier C composites (background | pasted mask | composite depth). Right: the three cat scene cases for cat2 (the background's own cat is the layout).

7 Training (train.py)

7.1 What is trained: LoRA

LoRA (via `peft`, `unet.add_adapter`) adds a low-rank update to each targeted weight matrix W: the layer computes $W x + (\alpha/r) B A x$, where A is $r \times d_{in}$ and B is $d_{out} \times r$. With $r = 16$ and $\alpha = 16$ the scale α/r is 1. A starts Gaussian and B starts at zero, so training starts exactly at the base model. Targets: `to_q, to_k, to_v, to_out.0` in every attention layer of the UNet (self-attention attn1 and cross-attention attn2 in all 16 transformer blocks): 128 adapted matrices, 3.19 M trainable parameters, versus ~860 M frozen UNet parameters. The VAE,

text encoder and ControlNet are frozen and get no LoRA. LoRA weights are kept in fp32, frozen weights in bf16.

7.2 Inputs cached once

- **Latents**: $z_{0,i}$ = VAE.encode(image$_i$).mean × 0.18215 for every training photo: posterior mean, no sampling, shape 4×64×64.
- **Text embedding**: CLIP last hidden state of the fixed prompt "a photo of sks <class>" (1×77×768). The prompt never changes during training.
- **Depth maps**: 3×512×512 in [0,1] on the GPU (pixel space; the ControlNet has its own encoder).
- **Latent masks**: mask area-downsampled to 64×64 (soft values at the border).
- After caching, the VAE and text encoder move to the CPU (they return only for validation).

7.3 One training step

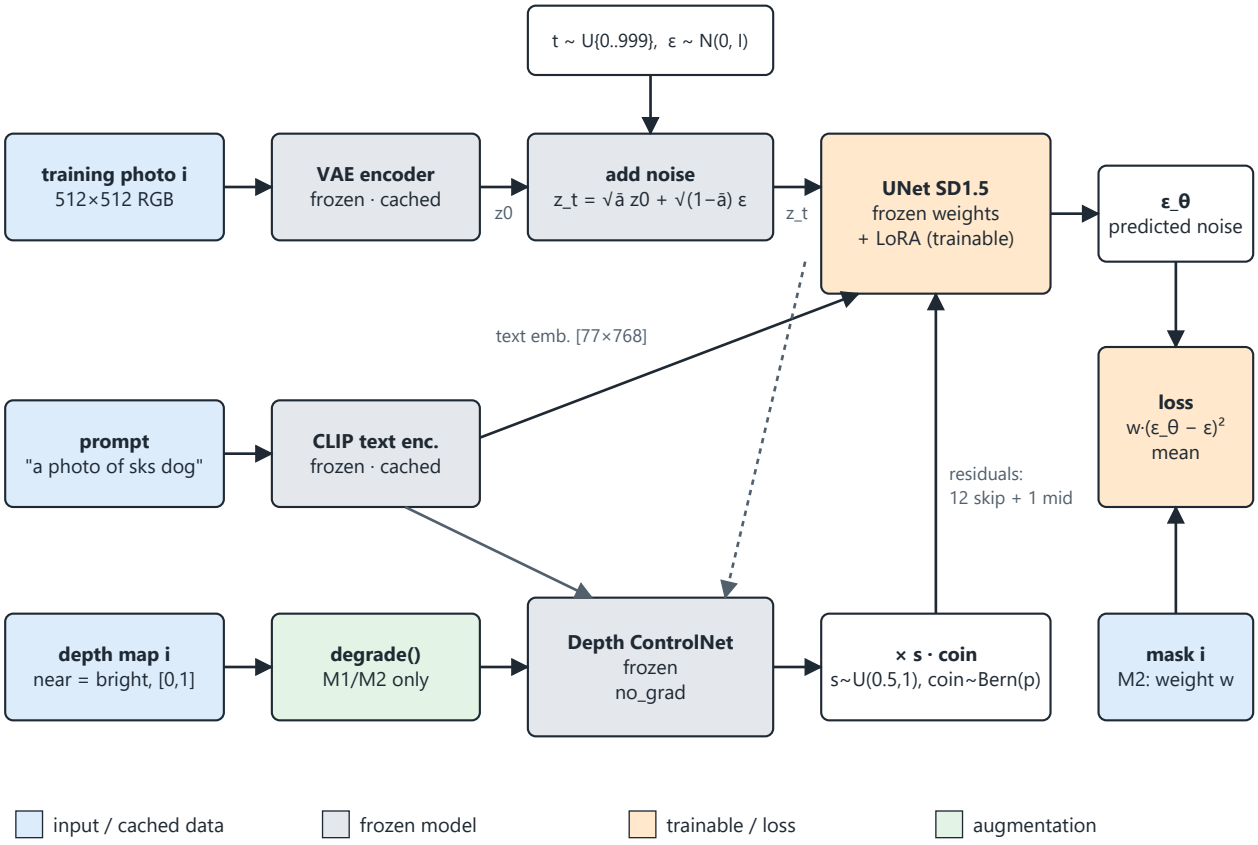


Figure 6. One micro-step. Grey: frozen. Orange: trainable LoRA and loss. The ControlNet branch is active with probability $p_{control}$ (0 for M0, 0.5 for M1/M2, 1 for M3).

For each micro-batch sample (batch size 1, two micro-batches per optimizer step):

- pick a random training photo i; draw noise ε ~ N(0, I) and timestep t uniform in {0, …, 999};
- noise the latent: $z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{(1 - \bar{\alpha}_t)}\, \varepsilon$ (DDPM forward process, `scheduler.add_noise`);
- **trick 1, control dropout**: flip a coin with probability $p_{control}$. If on: **trick 2**, degrade the depth map (M1/M2); **trick 3**, draw a strength s ~ U(0.5, 1); run the frozen ControlNet (no gradients) on (z_t, t, text embedding, depth). It returns 12 skip-connection residuals and 1 middle-block residual, which are multiplied by s;

- run the UNet with LoRA on (z_t, t, text embedding), adding the residuals to its skip connections and middle block (if control is on) → predicted noise ε_θ;
- **trick 4, masked loss** (M2 only): weight $w = m + (1 - m) \cdot 0.2$, where m is the latent mask (subject 1, background 0.2). Otherwise $w = 1$;
- loss = mean over 4×64×64 of $w \cdot (\varepsilon_\theta - \varepsilon)^2$, computed in fp32. SD1.5 predicts the noise ε (ε-prediction).

$$L = E_{i,\, t,\, \varepsilon,\, \text{coin},\, s,\, \text{degrade}} \left[w \cdot \| \varepsilon - \varepsilon_\theta(z_t, t, c_{text}, \text{coin} \cdot s \cdot \text{ControlNet}(z_t, t, c_{text}, \text{degrade}(\text{depth}_i))) \|^2 \right]$$

After two micro-batches the gradients (averaged) are clipped to norm 1.0 and an 8-bit AdamW step is taken (bitsandbytes; falls back to torch AdamW). The learning rate warms up linearly from 0 to 1e-4 over 50 steps, then stays constant. Only the LoRA parameters receive gradients: the ControlNet runs under `torch.no_grad()`, and gradient checkpointing is on in the UNet.

Implementation detail: per-sample strength. diffusers' ControlNet multiplies every residual by `conditioning_scale`. The code therefore runs it at 1.0 and multiplies the residuals by each sample's own s (0 if its coin was off). For batch size 1 this is identical to the brief's code; for larger batches it gives per-sample dropout and strength in one ControlNet call.

7.4 The four tricks

Trick	Setting	Problem it addresses
1 Control dropout	p_control = 0.5 (M1, M2)	Without it the LoRA becomes dependent on the control signal and text-only generation degrades (the same idea as ControlNet's 50% prompt dropout).
2 Degrade the depth map	affine + resample + blur, random each time	A perfectly aligned, same-photo map explains too much (even fine surface detail) and never occurs at test time. A coarser, slightly misaligned map explains pose/shape only, which pushes appearance into the LoRA.
3 Random strength	s ~ U(0.5, 1.0)	Makes the LoRA robust to the range of ControlNet strengths used at inference.
4 Masked loss	background weight 0.2 (M2)	Reduces the reward for memorizing the background.

degrade() (`boothnet/degrade.py`) is applied to the depth map only, never to the image or mask. Each call draws new magnitudes: (1) affine: translate ±4% of width/height, scale U(0.95, 1.05), rotate ±3°, with edge-value fill (`grid_sample`, border padding); (2) down-then-up bilinear resample by a factor from {1, 2, 4, 8}; (3) separable Gaussian blur with σ ~ U(0, 4) px. All parameters are logged per micro-step.

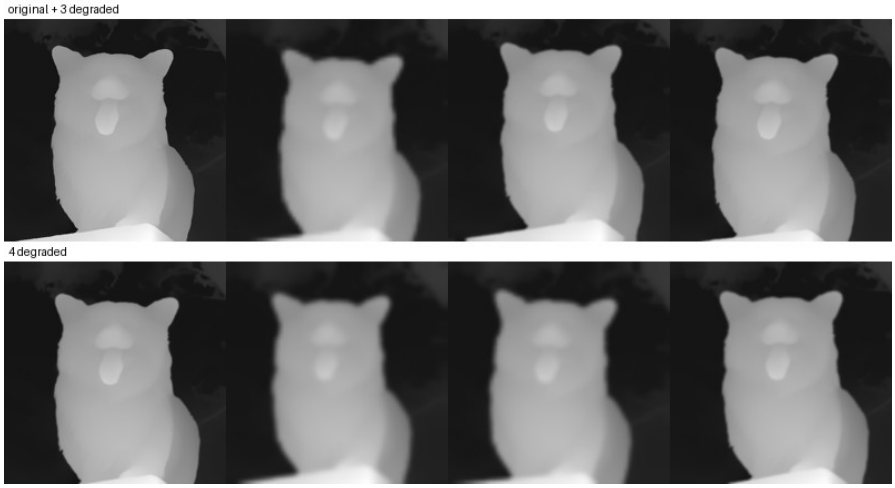


Figure 7. dog/01 depth map: original (top-left) and seven random degradations.

7.5 Hyperparameters (configs/train_base.yaml)

Parameter	Value	Parameter	Value
resolution	512	LoRA rank / alpha	16 / 16, Gaussian init
optimizer steps	1000 (×2 micro-batches)	LoRA targets	to_q, to_k, to_v, to_out.0
checkpoints	400, 600, 800, 1000 (+ final)	precision	bf16 frozen, fp32 LoRA + loss, autocast
learning rate	1e-4, constant, 50-step linear warmup	gradient checkpointing	on
optimizer	AdamW 8-bit, weight decay 1e-2	seed	0
batch / grad. accumulation	1 / 2	prior preservation	off (flag reserved, not implemented)
gradient clipping	max norm 1.0	validation	steps 0, 200, ..., 1000

7.6 Randomness and pairing

Two random generators are used: a Python `random.Random(seed)` for the photo index, control coin, strength and degradation, and a CUDA `torch.Generator(seed)` for the noise and timesteps. Every variant therefore sees the **same sequence of timesteps and noise** (verified for dog M0 vs M1). This is a paired design: loss differences between variants are not caused by different timestep draws. The photo indices diverge once M1 starts drawing control parameters.

7.7 Validation and outputs

At step 0 (untrained reference) and every 200 steps, with seed 0, UniPC 30 steps and CFG 7.5, the run renders (a) text-only "a photo of sks <class> on the beach" and (b) ControlNet (scale 0.8) with the subject's Tier A depth map and "a photo of sks <class>".

```
runs/<subject>/<variant>/
    checkpoint-400/ ... checkpoint-1000/, final/   pytorch_lora_weights.safetensors (diffusers format, 12.8 MB)
    config.yaml        fully resolved config (base + variant + CLI overrides)
    train_log.csv      per micro-step: step, micro, sample, img, t, loss, control_on, scale, lr,
                       deg_sigma, deg_factor, deg_tx, deg_ty, deg_scale, deg_rot
    val/step_XXXX.png, val/summary.png, loss.png
    summary.json       steps, peak VRAM, minutes, optimizer, LoRA size, train ids
```

Cost on the RTX 5070 Ti laptop: ~26 min per run (1000 steps plus 6 validations), 3.6 GiB peak VRAM.

8 Variants M0–M3 and evaluation methods

All variants use one script and differ only in their config (`configs/variants/`):

Variant	p_control	Degrade	Masked loss	Role	What it tests
M0	0	–	no	naive baseline: standard DreamBooth-LoRA	The reference. Combined with ControlNet only at inference = naive combination (B3).
M1	0.5	on	no	proposed	Does control in the loop make the LoRA learn appearance rather than layout?
M2	0.5	on	yes (0.2)	proposed + background disentanglement	Does down-weighting the background further reduce background memorization?

Variant	p_control	Degrade	Masked loss	Role	What it tests
M3	1.0	off	no	ablation	Always-on, perfect maps: expected to cause control dependence (poor text-only generation) and train/test mismatch. Keeps random strength.

At evaluation, a *method* is a LoRA variant combined with an inference mode:

Method	LoRA	ControlNet at inference	Prompt
B1: DreamBooth alone	M0	no	"a photo of sks <class>" + context
B2: ControlNet alone	none	yes	"a photo of a <class>" + context
B3: naive combination	M0	yes	personalized
Ours	M1, M2	yes	personalized
Ablation	M3	yes	personalized
Text-only sanity	M1, M2, M3	no	personalized (compare with B1)
Optional B4	BLIP-Diffusion + ControlNet	yes	only if time permits

9 Inference

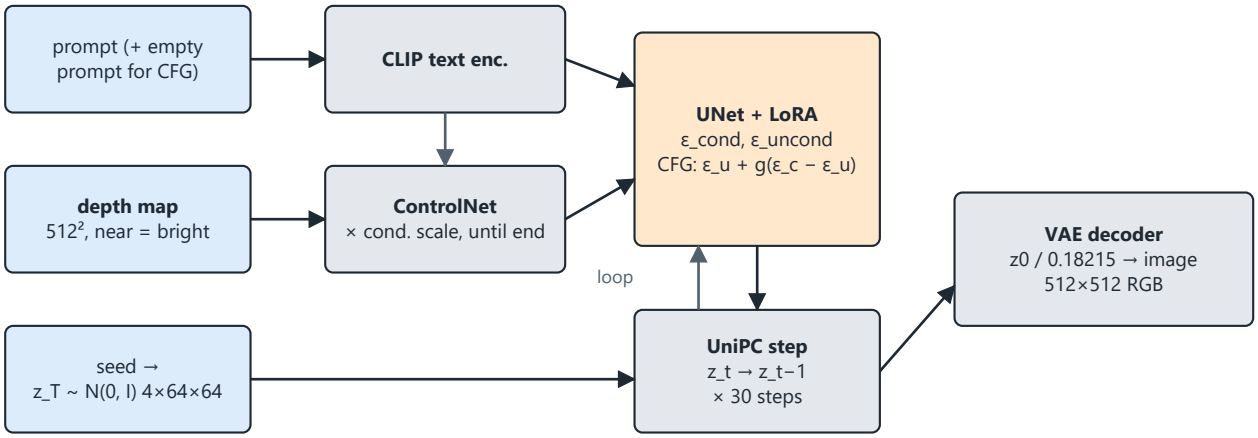


Figure 8. Inference with the ControlNet pipeline. The UNet runs twice per step (with the prompt and with the empty prompt) for classifier-free guidance.

9.1 Inputs

Input	Value used	Meaning
base model + ControlNet	SD1.5 + control_v11f1p_sd15_depth, fp16	StableDiffusionControlNetPipeline; safety checker off (false black images on benign subjects)
LoRA	runs/<subject>/<variant>/<checkpoint>	pipe.load_lora_weights(dir); pipe.unload_lora_weights() before loading another (none for B2)
prompt	personalized or class prompt + context	section 8; CFG also uses the empty prompt (no negative prompt)

Input	Value used	Meaning
image (control)	depth.png from the test bank	512^2 RGB, near = bright, values scaled to [0,1], no [-1,1] normalization
controlnet_conditioning_scale	0.8 (sweep 0.4–1.0 planned)	multiplier on all ControlNet residuals: layout strength
control_guidance_end	1.0 (sweep 1.0 / 0.7 planned)	fraction of steps with ControlNet active; 0.7 = released for the last 30% (details from the LoRA)
num_inference_steps	30	UniPC multistep sampler
guidance_scale	7.5	classifier-free guidance weight g
seed	0–3	torch.Generator("cuda").manual_seed(k): initial noise z_T; same seeds for every method (paired)

9.2 What happens inside

- The prompt and the empty prompt are encoded by CLIP (2 × 77 × 768).
- The depth image is resized to 512^2, converted to a [0,1] tensor and duplicated for the two CFG branches.
- Initial latent $z_T \sim N(0, I)$ of shape 4×64×64, drawn from the seed.
- For each of the 30 UniPC steps: the ControlNet computes residuals from (z_t, t, text, depth), multiplied by the conditioning scale and zeroed after control_guidance_end. The UNet with LoRA predicts ε_c (prompt) and ε_u (empty prompt). CFG combines them as $\varepsilon = \varepsilon_u + g\,(\varepsilon_c - \varepsilon_u)$, and UniPC updates z_t.
- The VAE decodes z_0 / 0.18215 into the 512×512 RGB output.

Text-only methods use `StableDiffusionPipeline` with the same model, scheduler, steps, CFG and seeds, and no depth map.

9.3 Outputs

One 512×512 RGB image per (method, test case, seed, scale, end). Phase 3's `generate.py` (planned) will write `outputs/<subject>/<method>/<tier>/<case_id>_s<scale>_e<end>_seed<k>.png` plus a manifest CSV, resumable. The exploratory sets so far are in `outputs/dog/_explore/`.

```
import torch
from diffusers import ControlNetModel, StableDiffusionControlNetPipeline, UniPCMultistepScheduler
from PIL import Image
cn = ControlNetModel.from_pretrained("lllyasviel/control_v11f1p_sd15_depth", dtype=torch.float16)
pipe = StableDiffusionControlNetPipeline.from_pretrained("stable-diffusion-v1-5/stable-diffusion-v1-5",
           controlnet=cn, dtype=torch.float16, safety_checker=None).to("cuda")
pipe.scheduler = UniPCMultistepScheduler.from_config(pipe.scheduler.config)
pipe.load_lora_weights("runs/dog/M1/checkpoint-400")
img = pipe("a photo of sks dog on the street of a mountain town",
           image=Image.open("data/test_bank/dog/C/city_roads/depth.png"),
           num_inference_steps=30, guidance_scale=7.5, controlnet_conditioning_scale=0.8,
           control_guidance_end=1.0, generator=torch.Generator("cuda").manual_seed(0)).images[0]
pipe.unload_lora_weights()
```

9.4 Planned evaluation (Phase 4)

Metric	How	Measures
DINO-masked ↑	segment the subject (same SubjectMasker), crop to the mask box, gray background, 224^2; mean pairwise cosine of DINO ViT-S/16 CLS embeddings vs training + held-out photos	identity (preferred)
CLIP-I-masked ↑	same crops, CLIP ViT-B/32 image embeddings	identity
DINO / CLIP-I full ↑	as in DreamBooth (no masking); rewards copying training backgrounds	comparability

Metric	How	Measures
CLIP-T ↑	image vs prompt with "sks " removed	prompt alignment
depth RMSE ↓	re-estimate depth (Depth-Anything-V2), least-squares scale + shift to the input map, RMSE on [0,1]	layout fidelity
mask IoU ↑ (B/C)	output mask vs case source_mask	placement / silhouette
background fidelity ↓ (C)	depth RMSE outside the source mask	scene followed?

Planned settings: seeds 0–3; scale sweep {0.4, 0.6, 0.8, 1.0} × control_guidance_end {1.0, 0.7} on dog and backpack (methods B2, B3, M1, M2) to choose one setting; then the full evaluation of all subjects, tiers and methods. The headline plot is identity (DINO-masked) vs layout (depth RMSE), one curve per method across the scale sweep (better = up and left).

10 Results so far (dog)

Run	Time	Peak VRAM	Control on	Mean loss (on / off)	Final step loss
dog / M0	25.6 min	3.63 GiB	0%	– / 0.127	0.042
dog / M1	26.3 min	3.64 GiB	49.3% of micro-steps	0.123 / 0.135	0.049

M1 statistics: mean strength s = 0.74; resample factors 1/2/4/8 drawn evenly (240/242/254/251 of 987); mean blur σ = 2.08 px; the four training photos were each used ~500 times. On the micro-steps where M1 had control on, M0's loss (same t and noise, but possibly a different photo) averaged 0.120 and M1's 0.123. Control in the loop does not visibly lower the denoising loss, and the loss curve (dominated by the random timestep) does not separate the variants.

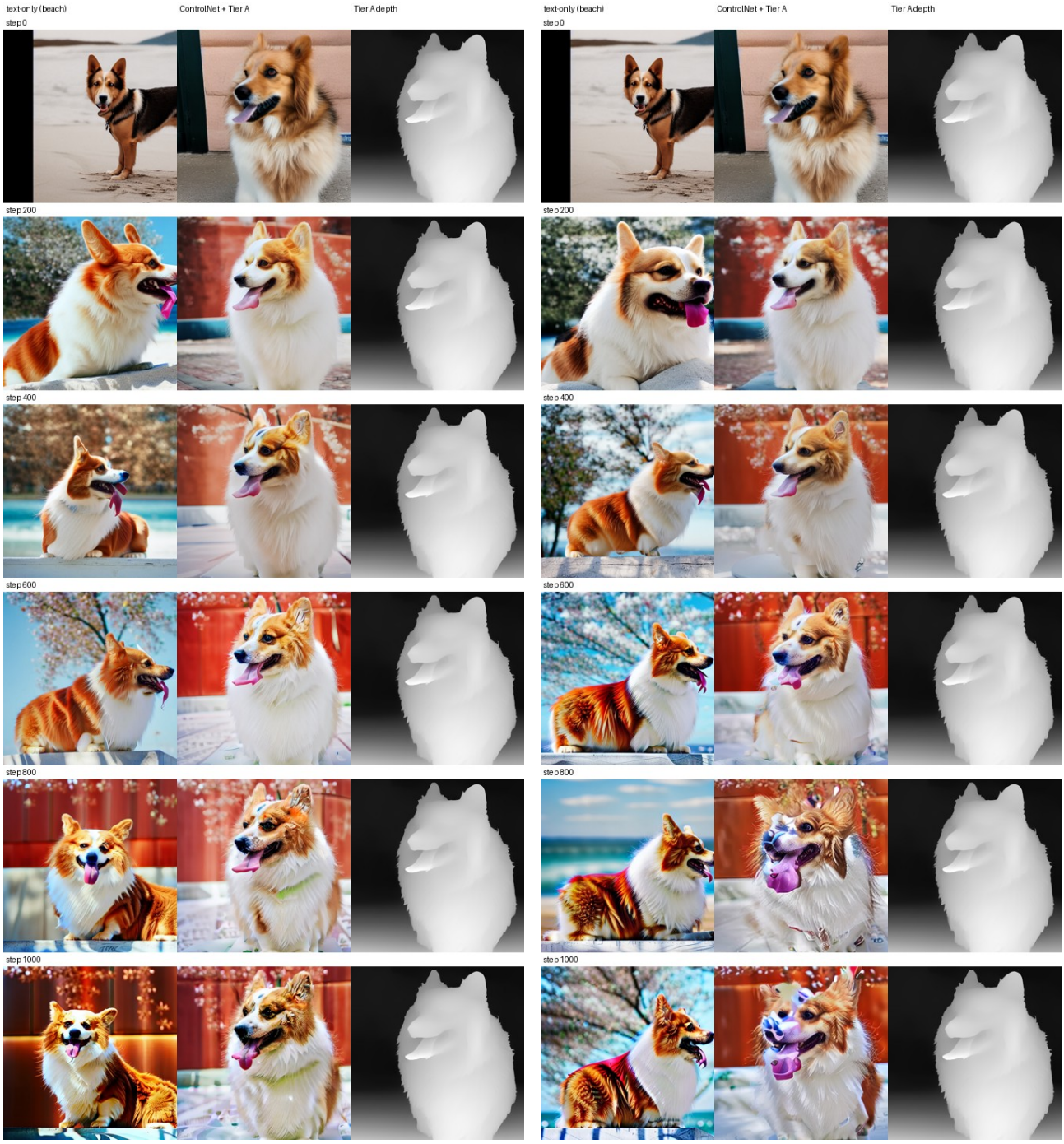


Figure 9. Validation images every 200 steps, M0 (left) and M1 (right): text-only "... on the beach" | ControlNet 0.8 + Tier A depth | the depth map.

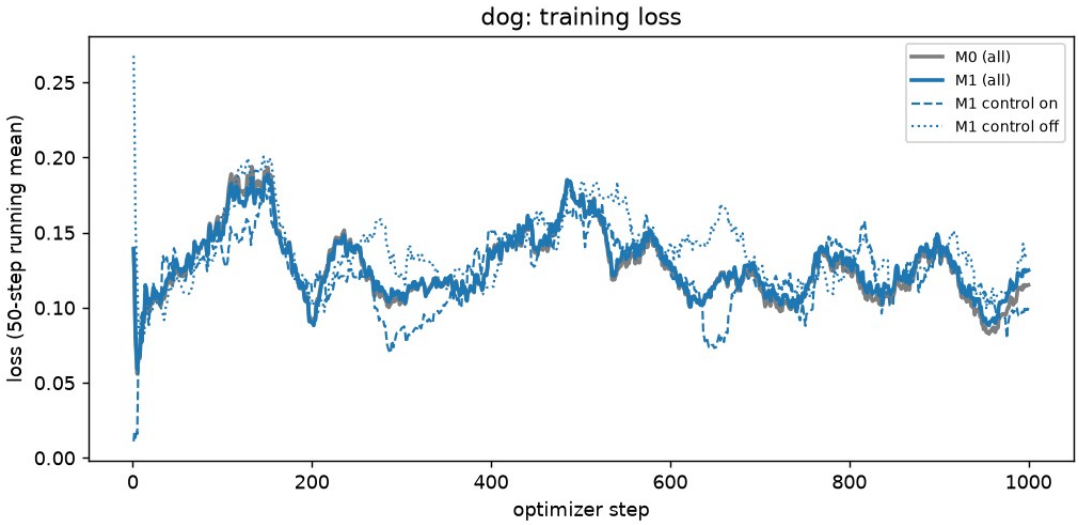


Figure 10. Training loss (50-step running mean). M1 split into control-on and control-off micro-steps.



Figure 11. Checkpoint comparison (rows: M0/M1 at 400, 600, 1000 steps). Columns: seed 0 and seed 1 × {ControlNet + Tier B dog2_00 (no context), ControlNet + Tier C beach1, text-only "on the beach"}.

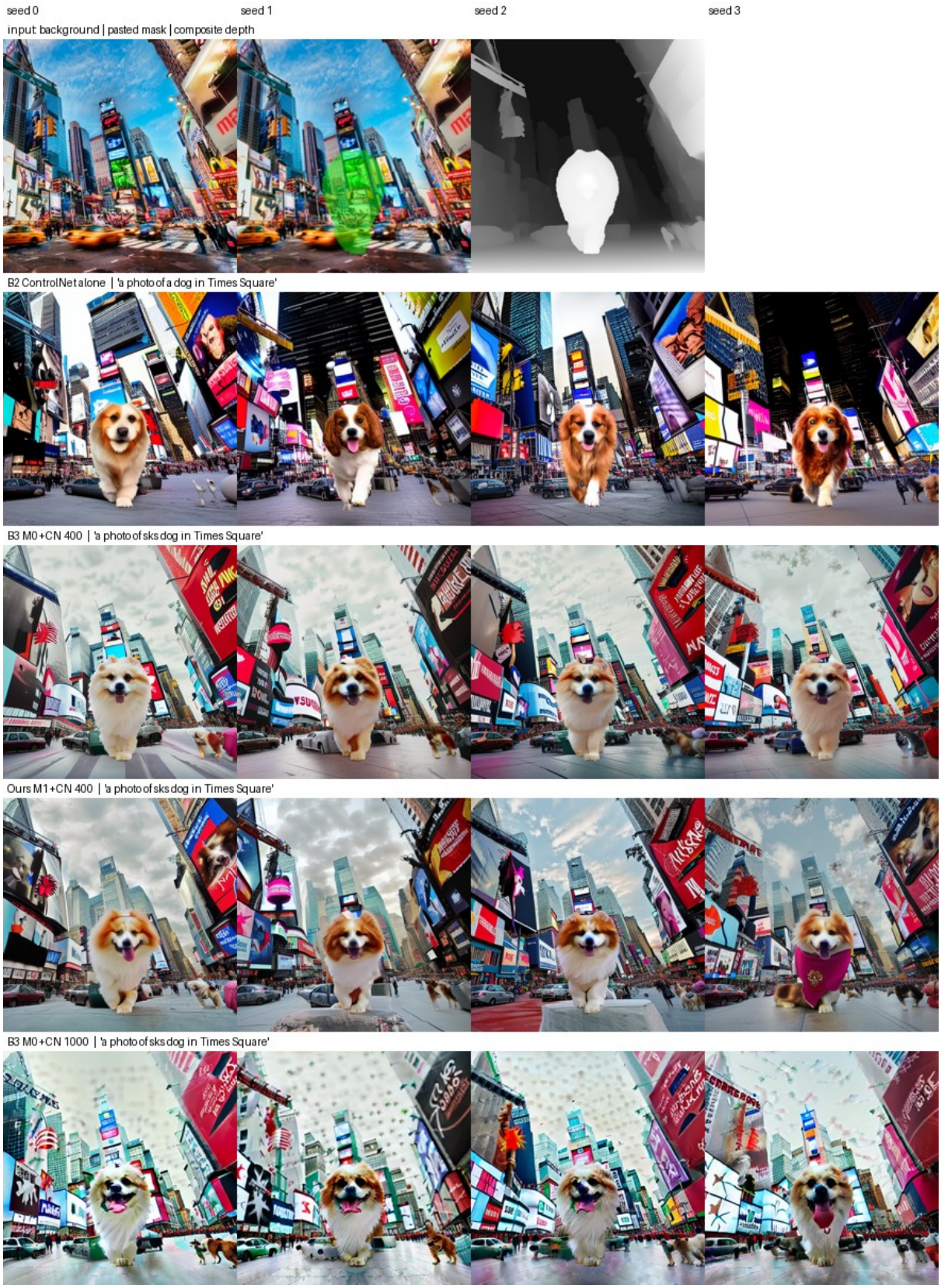


Figure 12. Times Square set, part 1: inputs (background, pasted mask, composite depth); B2 ControlNet alone; B3 M0+CN and Ours M1+CN at 400 and 1000 steps. Seeds 0–3 in columns.

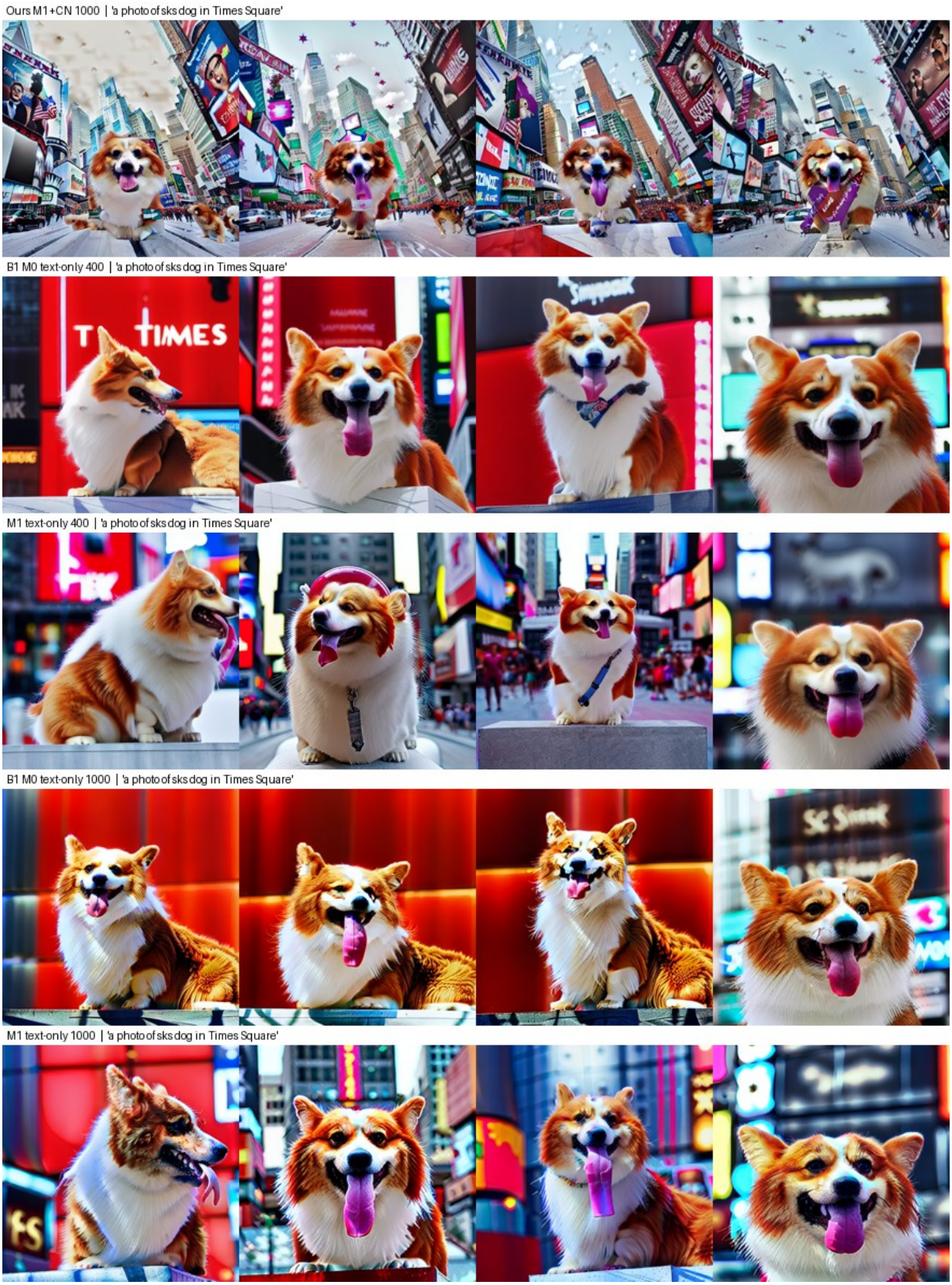


Figure 13. Times Square set, part 2: M1+CN at 1000 steps, then text-only B1 (M0) and M1 at 400 and 1000 steps.

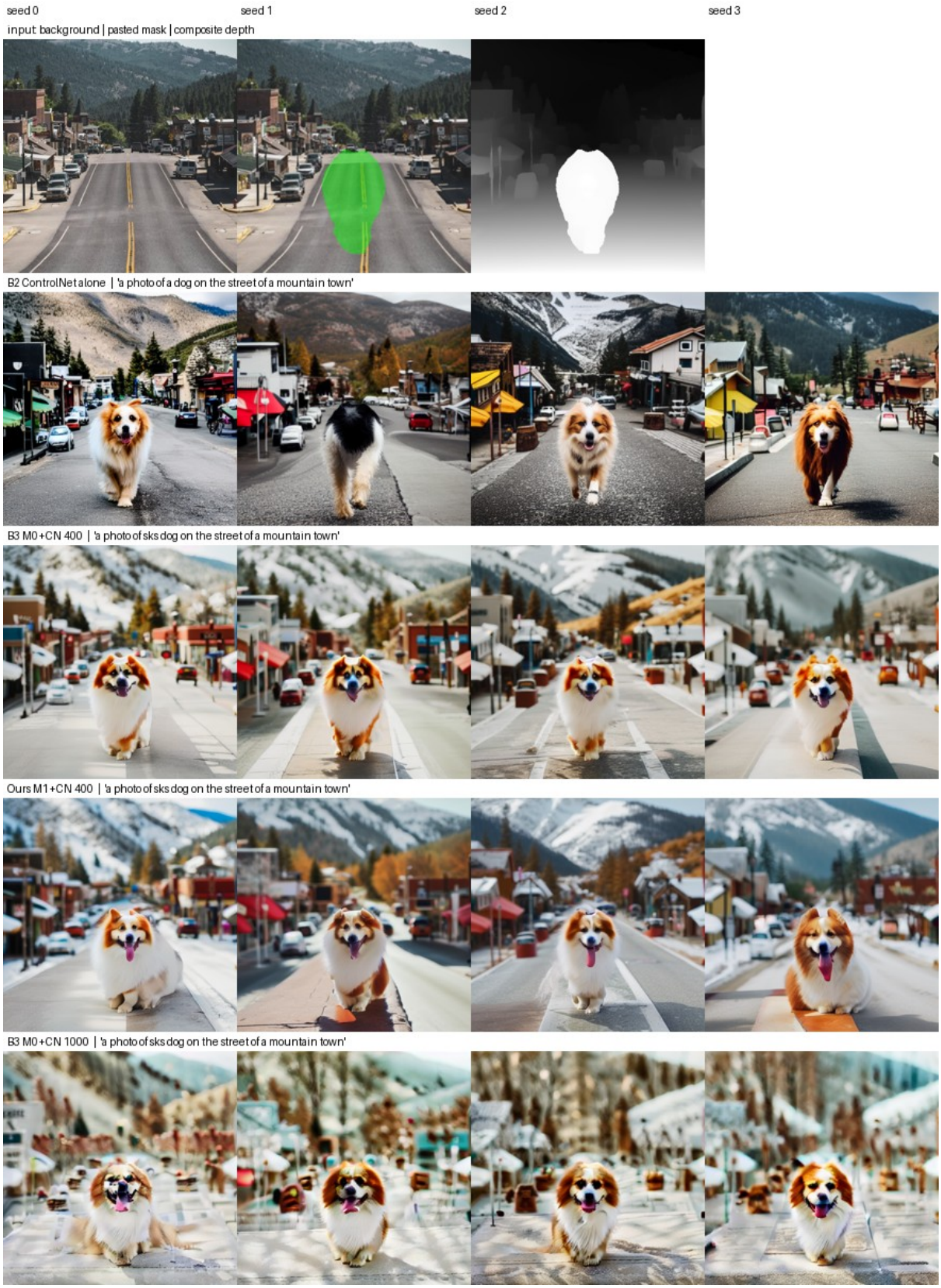


Figure 14. city roads ("on the street of a mountain town") set, part 1: inputs (background, pasted mask, composite depth); B2 ControlNet alone; B3 M0+CN and Ours M1+CN at 400 and 1000 steps. Seeds 0–3 in columns.



Figure 15. city roads ("on the street of a mountain town") set, part 2: M1+CN at 1000 steps, then text-only B1 (M0) and M1 at 400 and 1000 steps.

10.1 Observations

- **Identity** is learned by step 200 in both variants. Given a chow chow's depth (Tier B), both render the corgi in that pose.
- **Layout**: with ControlNet, all methods follow the composite layouts (scene perspective, dog at bottom-center in the pasted pose). B2 (no LoRA) is realistic but a generic dog with no identity.
- **Background memorization with ControlNet**: with no context in the prompt (Tier A/B), both M0 and M1 paint the training background (red wall, cherry blossoms) by step 400. With a background depth map and a context prompt (Tier C), both produce the requested scene. At checkpoint 400, M0+CN and M1+CN are hard to tell apart by eye over 4 seeds.
- **Text-only is where M1 differs most**: M0 reproduces training compositions (dog on a ledge, close-ups, the red wall in 3 of 4 seeds at step 1000 even for "in Times Square"). M1 at step 400 produces new poses (standing, walking, lying) in the requested scene. This supports the claim that control in the loop reduces pose and background memorization, at least for text-only generation.
- **Overtraining**: both variants over-saturate by step 1000. M1 degrades more: purple patches, doubled facial features with ControlNet at 800–1000 steps. M0+CN at 1000 damages the city_roads scene (washed out, road turns to texture). Checkpoint 400 looks best for both.
- These are qualitative impressions from 2–4 seeds on one subject. The metrics of Phases 3–4 are needed before drawing conclusions.

11 Decisions and deviations from the brief

Topic	Brief	What was done	Why
Masker	rembg or BiRefNet	per-category hybrid: Grounding-DINO + SAM (objects), BiRefNet ∩ detector box (animals)	salient masks included the person / saucer; SAM-from-box missed corgi bodies
SAM candidate	–	largest in-box candidate (IoU > 0.7), hole fill <0.1%	argmax-IoU dropped rc_car's driver
Cropping	center crop	strip black bars; subject-aware crop when non-square	baked-in letterbox bars in 4 files
rc_car Tier B	"a photo of a toy"	"a photo of a toy car", seed filter (object not cut off)	class noun too vague; SD close-ups unusable
Tier C pose source	Tier A or B	Tier A unless truncated, else first un-truncated Tier B	held-out dog/cat photos are cut off
Cat backgrounds	C-scene if subject inside	used as C-scene for cat2, depth only (no Canny)	user decision
Gradient clipping	not specified	max norm 1.0	diffusers DreamBooth-LoRA default
Per-sample scale	conditioning_scale= s	ControlNet at 1.0, residuals × s	identical for B=1; correct for B>1
M3 strength	not specified	keeps s ~ U(0.5, 1)	only dropout and degradation differ
Validation	every 200 steps	also at step 0	untrained reference
Prior preservation	flag for later	reserved, raises NotImplementedError	off by default anyway
API	torch_dtype=	dtype=	deprecated in diffusers 0.42 / transformers 5
onnxruntime	–	pinned 1.23.2 (CUDA 12)	1.31 is a CUDA-13 build and silently uses the CPU

12 Known issues and limitations

- **Overtraining at the default schedule.** 1000 steps × 2 micro-batches on 4 photos is ~500 passes per photo at LR 1e-4. Both variants over-saturate, M1 more. The checkpoint (400/600/800/1000) should be chosen in the Phase 3 sweep, the same step for every variant.
- **Tier A flatters every method.** Base SD1.5 with the dog's own depth map already produces a corgi-like dog (ears, tongue, pose). Depth carries silhouette identity, so Tier B/C matter more for H1.
- **Flat composite silhouettes.** Pasted subjects get depth in ~[0.9, 1] (feet near the camera), which loses internal 3-D structure. Raising y_bottom or rescaling the relief would keep more.
- **Pose sources are shared.** dog and dog6 use the same Tier C pose source (a chow chow), and the objects reuse their own held-out silhouette, so Tier C mixes "new scene" with "Tier A/B shape".
- **Small mask flaws.** rc_car antenna missing in 02/04 (the detector box excludes it); backpack 00/01 include a strap; masks use a 0.5 threshold with no edge refinement.
- **Same-image depth in training.** Even degraded, the training depth map always comes from the training photo itself. The model never sees the test situation (a subject's appearance + someone else's geometry) during training.
- **No context in training prompts.** "a photo of sks dog" gives the token no help separating subject from background (the red wall is part of "sks dog").
- **Identifier token.** "sks" is widely reported in the DreamBooth community to carry a prior in SD1.5 (a rifle model name). Not tested here.
- **Evaluation not implemented.** All claims so far are visual, from one subject and 2–4 seeds.
- **Different hardware.** The 22 remaining runs are planned on a server GPU; tiny numerical differences from the laptop dog runs are expected. Record server versions in NOTES.md.

13 Ideas for improvement

These are suggestions, not tested. Each changes the method or the brief's defaults, so it would be a separate, clearly labelled experiment.

Training schedule

- Lower LR (e.g. 5e-5) or fewer steps (400–600). Compare M0/M1 at matched steps. Cheap: the checkpoints already exist.
- Prior preservation (class images from base SD1.5) is the standard DreamBooth fix for over-saturation and language drift; the flag is reserved in train.py.

Make training look like testing

- **Background compositing in training** (as in BLIP-Diffusion): paste the masked subject onto random backgrounds, and build the matching composite depth map with the same code as Tier C. The background then stops being explained by "sks", and training uses the same kind of map as testing.
- **Stronger or geometric degradation**: elastic warps, morphological dilation of the subject region, or dropping the background depth (subject-only depth). This forces the LoRA to rely less on exact geometry.
- **Captioned training prompts**: "a photo of sks dog in front of a red wall" so the context words absorb the background (a known DreamBooth trick).

Control schedule

- Tune p_control (0.3 / 0.7) and the strength range. An idea: dropout per timestep range, with control only at high noise (layout is decided early, appearance late), mirroring control_guidance_end at inference.
- At inference, control_guidance_end < 1 (already in the sweep) leaves the last steps to the LoRA for details.

Evaluation and analysis

- More seeds (≥ 8) for the qualitative comparisons; report failure counts of the masker per method.
- Use the per-micro-step training log: loss vs timestep, control on/off at equal t (the RNG design makes this a paired comparison), and loss vs degradation strength.
- Optional: another control type (Canny edges) as an extension, once the depth study is done.

14 Code map, commands and configuration reference

File	Key contents
boothnet/preprocess.py	strip_borders, square_offset, square_crop_resize, crop_for_subject; DepthEstimator; Masker (BiRefNet session, context manager); GroundedSam (detect, segment_box with candidate rule); box_mask, fill_small_holes, filter_components; SubjectMasker (stage_torch, stage_salient, mask_batch)
boothnet/degrade.py	degrade(depth, cfg, rng) → (map, params): _affine (grid_sample, border), resample, _blur
boothnet/data.py	load_split, load_train_set (train photos only), encode_latents (posterior mean × 0.18215), masks_to_latent (area 64×64)
boothnet/models.py	load_components (frozen SD1.5 + ControlNet), add_lora (peft, fp32 params), save_lora (diffusers format), encode_prompt
boothnet/utils.py	ROOT, load_yaml/save_yaml/json, read_class_file, image_files, grid (contact sheets)
scripts/check_env.py	Phase 0: versions, one depth + ControlNet generation, peak VRAM
scripts/prepare_data.py	Phase 1 preprocessing (section 5)
scripts/build_test_bank.py	Tier A/B/C/C_bg (section 6): generate_sd_images, composite_depth, pick_pose_source
train.py	resolve_config (deep merge + CLI), training loop (section 7), validate, plot_loss
scripts/train_all.py	all subjects × variants as subprocesses; skips runs with final/; logs in runs/logs/
scripts/plot_losses.py	overlay loss curves of several variants

```
# environment (see README.md / README_SERVER_INSTRUCTIONS.md)
conda activate X:\Environments\BoothNet
python scripts/check_env.py
# Phase 1
python scripts/prepare_data.py [--subjects dog] [--overwrite]
python scripts/build_test_bank.py                  # tiers A + B
python scripts/build_test_bank.py --tiers C        # needs confirmed: true in configs/backgrounds.yaml
python scripts/build_test_bank.py --preview_c      # placement preview only
# Phase 2
python train.py --subject dog --variant M1 --max_steps 20   # smoke test -> runs/_smoke/
python train.py --subject dog --variant M1                  # full run
python scripts/train_all.py [--subjects ...] [--variants ...] [--dry_run]
python scripts/plot_losses.py --subject dog --variants M0 M1
```

Configuration files

File	Controls
configs/subjects.yaml	subjects (name, class, category, seg_prompt), resolution, identifier, crop options, masker thresholds (box 0.30, min_overlap 0.10, min_area 0.002, max_hole_frac 0.001)

File	Controls
configs/test_bank.yaml	Tier B sources per subject, SD seed search (sd_count, sd_max_seed, border margin), SD settings
configs/backgrounds.yaml	per background: type (composite / scene), context phrase, placement overrides; placement defaults; pose_source; bg_only_subjects; confirmed flag
configs/train_base.yaml	all training hyperparameters (7.5), trick settings (base = off), validation settings
configs/variants/M*.yaml	variant overrides: p_control, degrade.enabled, masked_loss

Further reading in the repository: PROJECT_BRIEF.md (the specification), NOTES.md (chronological log with every problem and fix), README.md and README_SERVER_INSTRUCTIONS.md (how to run).